

Multimedia Retrieval - Report

Rutger Vincken¹, Bar Melinarskiy¹, and Arie Klaver¹

¹ Graduate School of Natural Sciences, Utrecht University, the Netherlands

September 16, 2025

Contents

1	Introduction	2
2	Dataset	2
3	Mathematical Notations	4
4	Technical Prerequisites	4
4.1	Framework Architecture and Design Rationale	5
5	Step 1: Read and view the data	5
5.1	Data Format Processing and Geometric Representation	5
5.2	Interactive Visualization System	5
5.2.1	Wireframe Edge Visualization	6
5.3	3D Shape Viewer Web Application	6
5.4	Hierarchical Data Organization and Metadata Extraction	7
5.5	Performance and Usability	7
5.6	3D Visualization and Display Features	7
6	Step 2: Preprocessing and cleaning	8
6.1	Step 2.1: Analyzing a single shape	9
6.2	Step 2.2: Statistics over the whole database	9
6.3	Step 2.3: Resampling outliers	13
6.4	Step 2.4: Checking the resampling	13
6.5	Step 2.5: Normalizing shapes	15
7	Step 3: Feature extraction	19
7.1	Step 3.1: Full normalization	19
7.2	Step 3.2: 3D shape descriptors	19
8	Step 4: Querying	19
9	Step 5: Scalability	19
10	Step 6: Evaluation	19
	References	20

Abstract

TBD

1 Introduction

Content-based 3D shape retrieval addresses the fundamental challenge of identifying similar three-dimensional objects within large databases using geometric and topological properties rather than textual metadata. Given a query shape, such systems must rank database entries by similarity and return the most relevant matches, requiring sophisticated understanding of shape characteristics across different scales and representations.

In this work, we implement a complete 3D shape retrieval system using a collection of triangular meshes spanning diverse semantic categories and geometric complexities. Originating from the Princeton Shape Benchmark [1], the dataset includes everyday objects, vehicles, buildings, tools, and animal models, offering a broad test-suite for evaluating retrieval accuracy and computational performance.

The system was developed through six main stages: data visualization and preprocessing, geometric normalization, feature extraction, similarity metric design, spatial indexing, and performance evaluation. Each stage builds on the previous to form an end-to-end pipeline that allows querying a shape and retrieving the most similar objects from the database. The following sections describe these stages in detail and present the results of our evaluation. The full source code to our project is available via [GitHub](#).

2 Dataset

For this multimedia retrieval assignment, we utilized a comprehensive 3D shape database containing diverse geometric objects organized into semantic categories. The database comprises shapes sourced from the Princeton Shape Benchmark [1], offering a diverse range of 3D mesh objects suitable for content-based shape retrieval research. The complete data set used can be found [here](#).

The dataset comprises 2,483 3D mesh objects distributed across 69 distinct semantic categories, ranging from everyday objects like chairs, tables, and cups to more specialized items such as aircraft, musical instruments, and architectural elements. This diversity ensures comprehensive coverage of different shape characteristics, geometric complexities, and semantic domains essential for robust retrieval system evaluation. The category distribution ranges from 15 to 143 shapes per category, with an average of 36.0 shapes per category and a median of 27 shapes per category.

Each shape in the database is associated with a class label corresponding to its semantic category, derived from the original benchmark collections' folder organization structure. These class labels serve as ground truth for evaluation purposes in our content-based shape retrieval system. The shapes are stored in OBJ format [2], a widely-used standard for 3D mesh representation, containing vertex coordinates, face connectivity information, and associated geometric metadata.

The collection, as detailed in Table 1, exhibits significant variety in geometric complexity, with shapes averaging 5,908 vertices and 12,110 faces per object. This natural variation in mesh resolution and complexity provides an excellent testbed for evaluating the robustness and effectiveness of different shape descriptors and retrieval algorithms across objects of varying geometric detail and structural characteristics. Given the inherent variability in mesh resolution and geometric representation across the collection, a systematic preprocessing and normalization pipeline was essential to ensure consistent feature extraction and reliable retrieval performance. The comprehensive details of this multi-step process are presented in Sections 6 and 7.

Table 1: 3D Shape Dataset Composition

Category	Count	Avg Vertices	Avg Faces
AircraftBuoyant	16	2,509	4,925
Apartment	23	11,924	14,838
AquaticAnimal	29	4,216	8,136
Bed	27	5,357	9,687
Bicycle	26	15,402	30,099
Biplane	45	8,978	15,099
Bird	44	2,836	5,536
Bookset	20	880	4,541
Bottle	31	3,292	6,106
BuildingNonResidential	44	3,058	5,726
Bus	20	7,732	47,260
Car	89	11,170	20,282
Cellphone	20	9,793	18,375
Chess	42	3,837	7,636
City	27	5,955	8,765
ClassicPiano	20	3,035	16,214
Computer	33	3,118	5,780
ComputerKeyboard	20	4,308	22,554
Cup	30	3,695	7,014
DeskLamp	31	4,718	8,384
DeskPhone	20	16,459	29,363
Door	44	1,151	2,171
Drum	20	2,692	15,600
Fish	20	3,898	6,529
FloorLamp	28	6,284	11,059
Glasses	20	5,161	9,562
Guitar	20	9,831	17,443
Gun	58	4,391	8,476
Hand	37	6,380	12,715
Hat	15	2,339	4,552
Helicopter	51	5,772	10,804
House	46	7,485	15,064
HumanHead	51	4,085	13,916
Humanoid	142	4,240	8,418
Insect	66	3,698	7,071
Jet	143	1,142	2,098
Knife	27	650	2,739
MilitaryVehicle	20	14,750	24,322
Monitor	20	7,513	12,996
Monoplane	20	1,644	2,921
Motorcycle	26	14,202	25,867
Mug	26	5,909	11,624
MultiSeat	45	5,096	9,451
MusicalInstrument	21	8,074	15,687
NonWheelChair	60	2,264	4,108
PianoBoard	20	2,542	14,883
PlantIndoors	65	17,213	10,505
PlantWildNonTree	27	7,688	11,221
Quadruped	57	5,240	9,504
RectangleTable	64	817	1,615
Rocket	20	7,092	4,014
RoundTable	37	1,848	3,433
Shelf	43	1,289	1,686
Ship	63	5,076	8,690
Sign	15	1,402	2,703
Skyscraper	20	25,559	25,817
Spoon	20	959	3,291
Starship	31	5,596	11,386
SubmachineGun	20	5,295	9,687
Sword	53	4,529	8,701
Tool	44	2,951	5,827
Train	30	5,273	25,320
Tree	47	15,587	13,812
Truck	27	7,176	31,413
TruckNonContainer	20	5,646	33,820
Vase	20	971	1,936
Violin	20	3,357	20,536
Wheel	17	1,192	2,218
WheelChair	20	12,465	22,059
Total	2483	5,908	12,110

3 Mathematical Notations

The following notations are used throughout this report:

- $V = \{v_1, v_2, \dots, v_n\}$: Set of mesh vertices, where each $v_i \in \mathbb{R}^3$.
- $F = \{f_1, f_2, \dots, f_m\}$: Set of mesh faces, where each f_j is a sequence of vertex indices.
- $M = (V, F)$: A mesh, consisting of vertices and faces.
- $\mathbf{n}_f$: The normal vector of face f .
- $\mathbf{n}_i$: The normal vector at vertex v_i , computed as the average of adjacent face normals.
- E : Set of unique mesh edges, extracted from the faces.
- $c \in \mathcal{C}$: The semantic class label of the mesh, where $\mathcal{C}$ is the set of all possible classes.
- $|V| = n$: The number of vertices in the mesh.
- $|F| = m$: The number of faces in the mesh.
- $\tau(f_j)$: The type of face f_j (triangle, quadrilateral, or polygon).
- $\mathcal{T}(M) = \{\tau(f_j) \mid f_j \in F\}$: The set of face types present in the mesh.
- $B(M) = [\mathbf{v}_{\min}, \mathbf{v}_{\max}]$: The axis-aligned bounding box of M , where $\mathbf{v}_{\min} = (\min_i v_i^x, \min_i v_i^y, \min_i v_i^z)$ and $\mathbf{v}_{\max} = (\max_i v_i^x, \max_i v_i^y, \max_i v_i^z)$.
- $\mathbf{s} = (s_x, s_y, s_z)$: Side lengths of the bounding box, with $s_x = v_{\max}^x - v_{\min}^x$, etc.
- $s_{\max} = \max(s_x, s_y, s_z)$: Longest side length of the bounding box.
- $\alpha = \frac{1}{s_{\max}}$: Uniform scaling factor ensuring that the longest side of the bounding box is mapped to 1.0.
- $\mathbf{b}(M)$: The mass barycenter of mesh M , computed as the area-weighted average of triangle centroids.

4 Technical Prerequisites

The following technical prerequisites were required for the development and execution of our 3D shape retrieval system:

- **Programming Languages:** Python 3.10
- **Third-party Libraries:**
 - ☐ Dash – web application framework
 - ☐ Plotly – interactive 3D visualization
 - ☐ Pandas – data manipulation and analysis
 - ☐ NumPy – numerical computations
- **Development Tools and Frameworks:**
 - ☐ Visual Studio Code – primary IDE
 - ☐ Git – version control
 - ☐ Anaconda – environment management

4.1 Framework Architecture and Design Rationale

The viewer app was developed as a web-based application, using Dash [3] to manage reactive interactions and Plotly [4] to provide efficient 3D visualization capabilities. The selection of this technology stack was guided by three requirements:

1. Real-time interactive exploration of the large mesh dataset.
2. Cross-platform accessibility without the need for specialized 3D software.
3. Seamless integration with Python-based scientific computing tools for efficient data processing, such as Pandas [5] and NumPy [6].

The framework architecture follows a model–view–controller paradigm [7], which separates data processing (model), user interface (view), and interaction logic (controller). This separation improves maintainability and supports extensibility for the integration of subsequent retrieval components.

5 Step 1: Read and view the data

Effective retrieval requires a clear understanding of the dataset at hand. To address this, our system begins with the development of an exploration and visualization framework tailored to 3D meshes. The following subsections provide a detailed breakdown of the application’s implementation.

The purpose of this stage is to establish a comprehensive environment for systematic analysis of the mesh datasets. Interactive visualization is a central component in multimedia retrieval, as it enables researchers to assess data distributions, identify potential preprocessing challenges, and validate algorithmic decisions.

5.1 Data Format Processing and Geometric Representation

Our system processes triangular mesh data stored in the Wavefront OBJ format, which has become the de facto standard for 3D model interchange due to its simplicity and widespread software support [2]. The OBJ format represents 3D objects through two primary components: vertex coordinates defining spatial geometry and face definitions specifying triangular connectivity.

We implemented a custom `OBJParser` class that addresses several technical challenges inherent in OBJ processing:

- **Index Convention Translation:** OBJ files use 1-based indexing while NumPy arrays require 0-based
- **Polygon Triangulation:** Automatic conversion of quad faces to triangles through diagonal splitting, ensuring consistent triangular mesh representation required for downstream processing
- **Robust Parsing:** Error handling for malformed geometry definitions and missing vertex references

The parser extracts vertex coordinates $V = \{v_1, v_2, \dots, v_n\}$ where $v_i \in \mathbb{R}^3$ and face connectivity $F = \{f_1, f_2, \dots, f_m\}$ where each face f_j contains three vertex indices, forming the fundamental mesh representation $M = (V, F)$ used throughout our retrieval pipeline.

5.2 Interactive Visualization System

The visualization component leverages Plotly’s WebGL-based 3D rendering engine, which provides hardware-accelerated graphics suitable for real-time interaction with complex meshes [8]. Our implementation incorporates several advanced rendering techniques:

To visualize 3D shapes in our web application, we use the `textitMesh3d` object from the Plotly Graph Objects library [9]. This tool allows us to render interactive 3D meshes directly in the browser, providing smooth rotation, zooming, and lighting effects. By leveraging Plotly’s capabilities, we can display complex surfaces and wireframes in a user-friendly way.

5.2.1 Wireframe Edge Visualization

Our optional wireframe overlay uses an efficient edge extraction method that avoids duplicate edges by storing them in a set. For each face f in the set of faces F , we iterate over its edges. Each edge is represented by a pair of vertex indices, sorted to ensure consistency. We add each sorted edge to the set E , which automatically removes duplicates.

This set-based approach guarantees $O(F)$ complexity for edge extraction, where F is the number of faces. The resulting set contains E unique edges. This significantly reduces rendering overhead compared to naive enumeration of all face edges [10, pp. 17–19].

Algorithm 1: Efficient Wireframe Edge Extraction

Input: Set of faces F (each face is a list of vertex indices)

Output: Set of unique edges E

```

1  $E \leftarrow \emptyset$ ; // Initialize empty edge set
2 foreach face  $f \in F$  do
3   for  $i \leftarrow 0$  to  $|f| - 1$  do
4      $v_1 \leftarrow f[i]$ ;
5      $v_2 \leftarrow f[(i + 1) \bmod |f|]$ ;
6      $edge \leftarrow \text{sort}(v_1, v_2)$ ; // Sort vertex indices for consistency
7      $E \leftarrow E \cup \{edge\}$ ; // Add edge to set (deduplication)
8   end
9 end
10 return  $E$ 

```

5.3 3D Shape Viewer Web Application

Our interactive web application, shown in Fig. 1, enables users to explore 3D models directly in their browser. The figure displays an example visualization of a bird mesh. Below, we describe the main components and design choices that support usability, performance, and data quality.

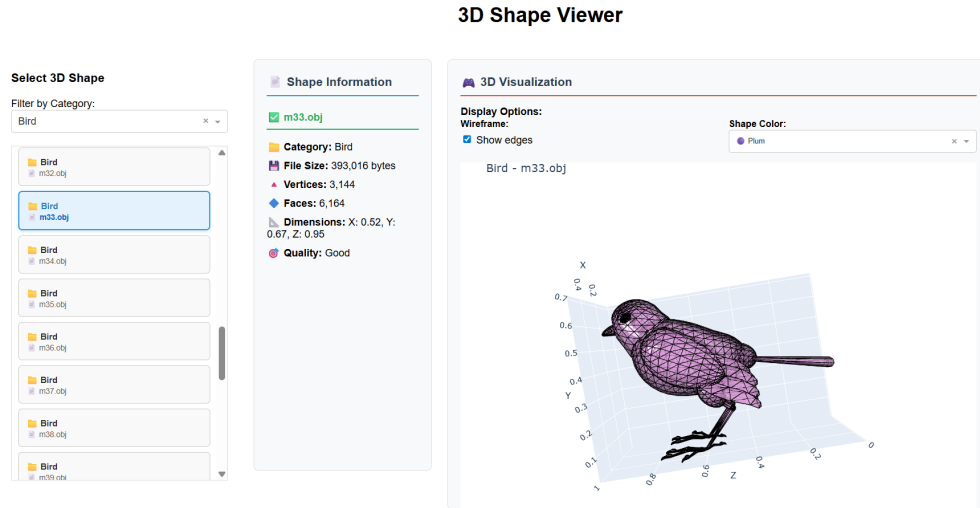


Figure 1: **Initial version of interactive user interface for 3D shape retrieval.** Users can filter the object list by category, such as “Bird,” or scroll through the complete list. Selecting a model reveals detailed information, including file size, vertices, faces, dimensions, and quality, along with an interactive 3D visualization. Display options such as wireframe edges and customizable colors support both structural inspection and visual comparison of shapes.

5.4 Hierarchical Data Organization and Metadata Extraction

To facilitate systematic exploration and evaluation, we utilized the organization of the 3D models into a hierarchical directory structure by category. The application automatically scans these folders, identifies available models, and extracts key metadata such as vertex and face counts, file size, and bounding box dimensions. This metadata is presented to users for quick assessment and is also used to label models as high or low resolution, supporting both user selection and algorithmic preprocessing. As shown in Fig. 2, users can filter models by category to quickly focus on the objects they wish to view.

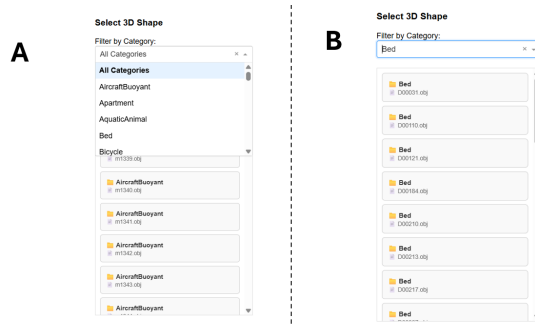


Figure 2: **Filter by category.** (A) The user can filter by a specific category or display objects from all categories. (B) For example, we filtered on all objects of beds.

5.5 Performance and Usability

To ensure a smooth user experience, our application is optimized for performance and reliability. Model files are loaded and parsed for display only when selected, which keeps memory usage low and startup times fast. Client-side handling of simple interactions, such as file selection highlighting, provides immediate feedback. Robust error handling further enhances usability by notifying users of any issues during file loading.

5.6 3D Visualization and Display Features

The viewer enables users to interactively explore 3D models using Plotly’s **Mesh3d** object. Users can rotate, zoom, and pan the models, as well as customize display options such as mesh color, wireframe edges, and smooth shading. The smooth shading feature allows users to toggle between flat and smooth surface rendering for enhanced geometric inspection (see Fig. 4). These features support both visual comparison and structural inspection, making it easier to understand the geometry and quality of each shape. These capabilities are demonstrated in Figures 3, 5, and 4.

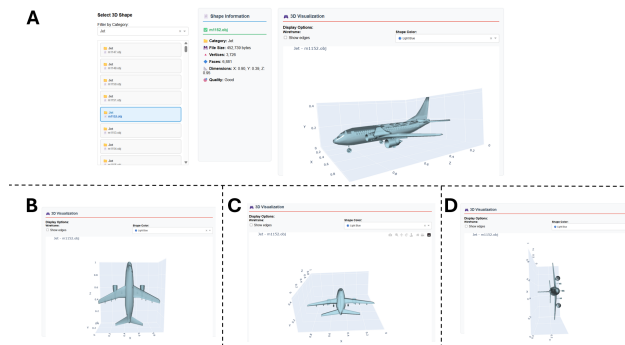


Figure 3: **Rotation demonstration.** (A) Example display of a plane object and its metadata; (B–D) the same object shown at different rotation angles.

To enable the user to control the shape’s shading, we utilized the *flatshading* parameter of Plotly’s Mesh3d object. This parameter controls whether normal smoothing is applied to the mesh: when enabled, the mesh is rendered with flat, angular faces for a low-poly appearance; when disabled, smooth shading is used, resulting in a visually softer surface with interpolated lighting across vertices. An example of this feature can be seen in Fig. 4.

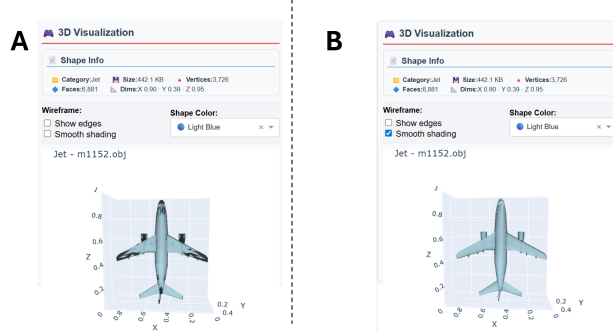


Figure 4: **Smooth shading demonstration.** (A) Example display of a plane object and its metadata using Plotly’s default flat shading, where each face is rendered with a uniform color and sharp edges are visible. (B) The same object shown with smooth shading enabled, resulting in a visually smoother surface appearance as lighting is interpolated across vertices. This feature allows users to toggle between flat and smooth shading in the viewer for enhanced geometric inspection and comparison.

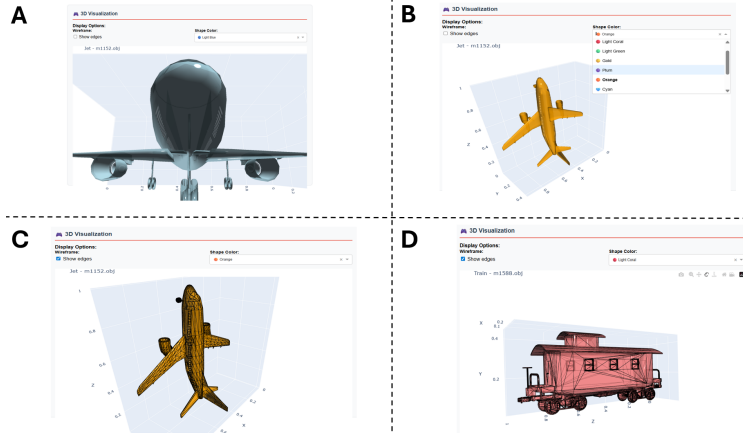


Figure 5: **Display features demonstration.** (A) Zooming in on the same plane object from Fig. 3. (B) Changing the object’s color from blue to orange. (C) Showing the plane’s edges by enabling the ”Show edges” checkbox above the viewer. (D) Displaying a pink train object with its edges visible, illustrating both color change and edge visualization.

This application streamlines the process of inspecting and comparing 3D models, providing both visual feedback and essential metadata to support further analysis and retrieval experiments.

6 Step 2: Preprocessing and cleaning

The dataset contains a large variety of 3D objects that differ significantly in complexity, resolution and overall quality. Before features can be reliably extracted and used for retrieval, it is essential to ensure that

all shapes satisfy certain quality requirements. Poorly sampled meshes, inconsistent scaling, or misaligned objects may bias the descriptors extracted later on, which in turn reduces the effectiveness and fairness of the retrieval system.

In this section, we analyze the structural characteristics of all shapes in the database and identify common irregularities such as extreme outliers in the number of vertices and faces, as well as variations in scale and positioning. By collecting global statistics over the dataset, we obtain a better understanding of its variability and can determine which objects require resampling or refinement.

Next, we apply a set of preprocessing operations to improve dataset consistency. These operations include mesh simplification for overly large shapes, refinement for undersampled ones, and normalization to standardize the position and scale of each object. The resulting database forms a more balanced and homogeneous foundation for the subsequent feature extraction stage, ensuring that differences in descriptors reflect meaningful shape characteristics rather than artifacts of the raw input data.

6.1 Step 2.1: Analyzing a single shape

Before starting the preprocessing and cleaning of the dataset, we first obtain an overview of the initial variability of the objects. For this purpose, we implemented a simple analysis tool that reports, for each shape $M = (V, F)$, the following properties:

- The semantic class label $c \in \mathcal{C}$ of the shape.
- The number of vertices $|V|$ and the number of faces $|F|$.
- The set of face types $\mathcal{T}(M)$, which indicates whether the shape consists of only triangles, only quadrilaterals, or a mixture of polygon types.
- The axis-aligned bounding box $B(M) = [\mathbf{v}_{\min}, \mathbf{v}_{\max}]$, defined by the minimum and maximum coordinates among all vertices.

This information provides a concise summary of the raw dataset characteristics and highlights potential sources of variability that may need to be addressed in subsequent preprocessing steps.

The tool operates by iterating over all files in the dataset in `.obj` format. In this format, each line encodes either a vertex position (`v`), a vertex normal (`vn`), or a face (`f`). During parsing, each vertex is added to the set V , and each face to the set F , while ignoring additional attributes such as normals and texture coordinates. Once all lines have been processed, we directly obtain $|V|$ and $|F|$. The bounding box $B(M)$ is computed by taking the coordinate-wise minima and maxima of the vertices. Face types are determined by checking the number of vertices in each face, distinguishing between triangles, quadrilaterals, and higher-order polygons.

The extracted statistics are written to a comma-separated values (CSV) file, producing one record per shape. In the next step, we will aggregate these records to obtain statistics over the entire dataset.

6.2 Step 2.2: Statistics over the whole database

Having obtained the individual characteristics of each object, we now analyze their distribution across the entire dataset. We start by examining the distribution of shapes across the different classes. The resulting histogram is shown in Figure 6.

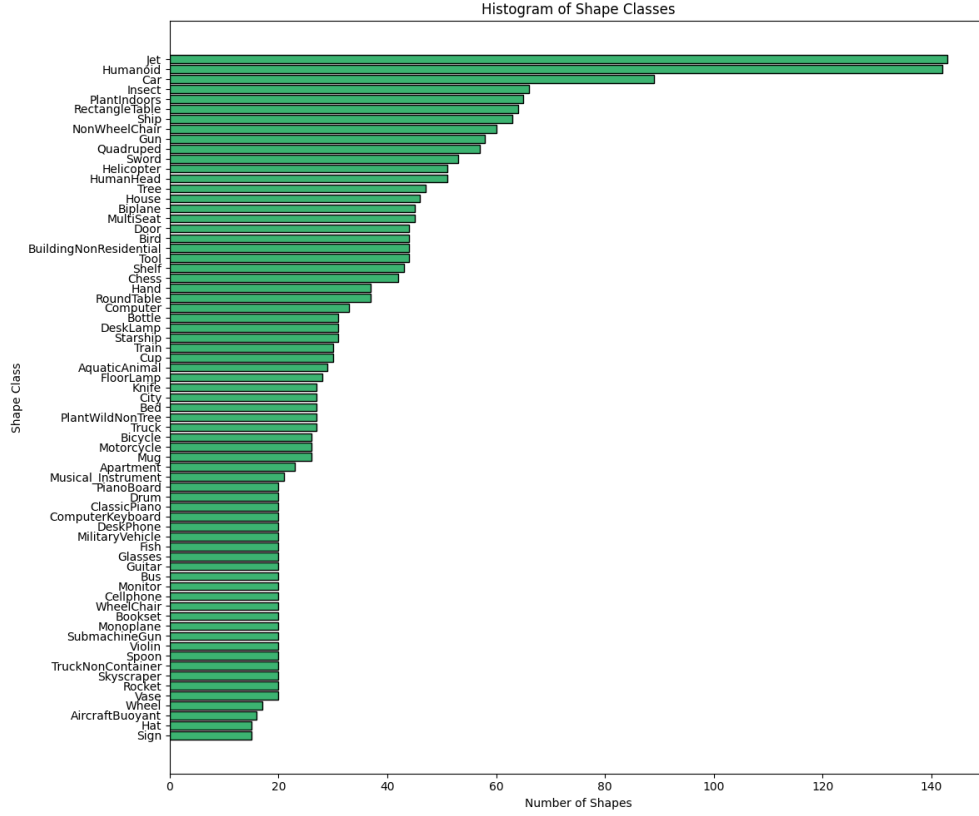


Figure 6: **Distribution of shape classes.** The histogram shows the number of shapes per class in the dataset. The vertical axis indicates the shape classes, while the horizontal axis represents the number of shapes in each class.

From the histogram, we observe that most classes contain between 20 and 50 shapes. There are a few outliers with either fewer shapes or significantly more. Notably, the *jet* and *humanoid* classes contain over 140 shapes each, representing the largest class sizes in the dataset.

Now that we have an overview of the distribution of shapes across classes, we next examine the vertex counts of the shapes. To visualize this, we plot a histogram of the vertex counts, shown in Figure 7. The histogram spans the range from 0 to 100,000 vertices and is divided into 30 uniform bins, each covering approximately 3,333 vertices.

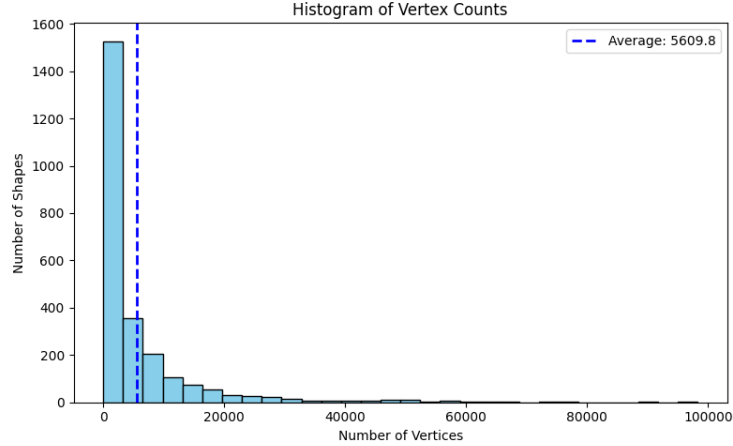


Figure 7: **Distribution of vertex counts.** Histogram of the number of vertices per shape. The horizontal axis shows vertex count (in bins of $\sim 3,333$ vertices) and the vertical axis shows the number of shapes in each bin. The dotted line indicates the mean vertex count.

We observe that most shapes fall into the lower bins, with vertex counts below 3,333. The mean vertex count of 5610 lies outside this first bin, indicating a skew towards higher counts. In addition, there are several clear outliers, with vertex counts exceeding 90,000.

The average shape, based on vertex count, is `D00926.obj`, shown in Figure 8. We also show several outliers.

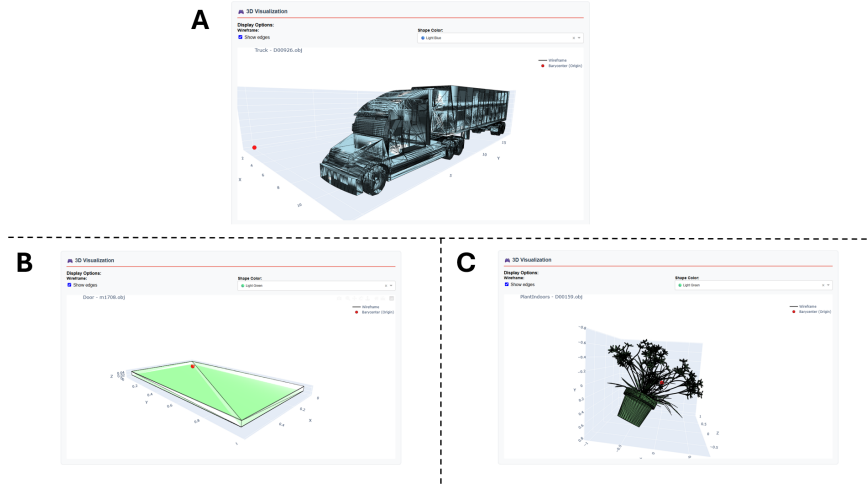


Figure 8: **Average shape and outliers for number of vertices.** (A) The average shape in the dataset has 5,610 vertices. The shape closest to this average is object `D00926.obj` of the *Truck* class, with 5,602 vertices. (B) The shape with the lowest number of vertices is *Door* object `m1708.obj`, with 16 vertices. (C) The shape with the highest number of vertices is *PlantIndoors* object `D00159.obj`, with 98,256 vertices.

We also examine the face count of the shapes. We again visualize this by plotting a histogram, this time of the face counts, as shown in Figure 9. The histograms spans the range from 0 to 140,000 faces and is divided into 30 uniform bins, each covering approximately 4,667 faces.

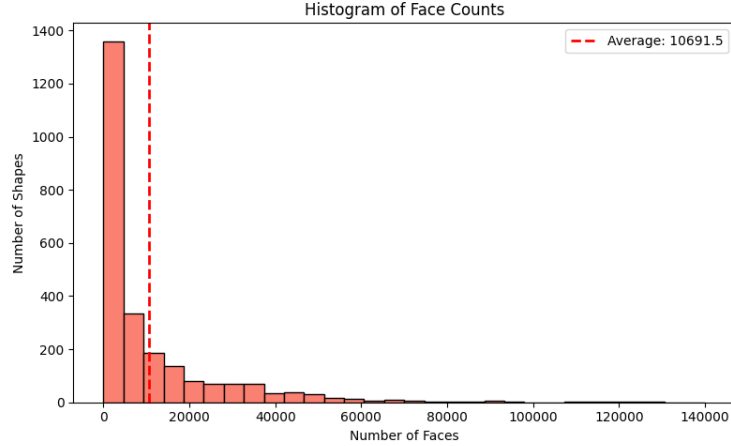


Figure 9: **Distribution of Face counts.** Histogram of the number of faces per shape. The horizontal axis shows face count (in bins of $\sim 4,667$ vertices) and the vertical axis shows the number of shapes in each bin. The dotted line indicates the mean face count.

We observe a similar distribution as with the vertex count, with a large spike in the lowest bin and a skew towards higher counts. The average however, lies higher then for the vertex count, at 10,692 faces. We also see similar outliers, this time ranging up to 130,000.

The average shape, based on the face count, is `m1120.obj`, shown in Figure 10. We also show several outliers.

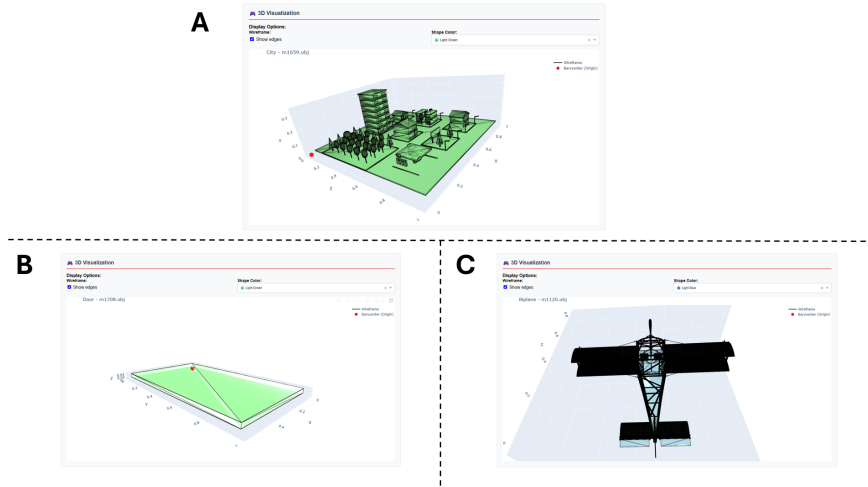


Figure 10: **Average shape and outliers for number of faces.** (A) The average shape in the dataset has 10,692 faces. The shape closest to this average is object `m1659.obj` of the *City* class, with exactly 10,692 faces. (B) The shape with the lowest number of faces is *Door* object `m1708.obj`, with 16 faces. (C) The shape with the highest number of faces is *Biplane* object `m1120.obj`, with 129,881 faces.

We also inspected the face types of the objects in the dataset, classifying them as consisting of only triangles, only quadrilaterals, or a mixture of polygon types. We found that all objects consist solely of triangular faces, so no further processing was required in this regard.

6.3 Step 2.3. Resampling outliers

From the obtained statistics it has become clear that the dataset contains a wide variety of shapes. To ensure fair and unbiased feature extraction and object retrieval, it is necessary to reduce the influence of extreme outliers in terms of vertex count. For this reason, we apply a remeshing step that regularizes the mesh resolution across the dataset. This ensures that all shapes are represented at a comparable level of detail, making subsequent analysis and retrieval more consistent and reliable.

For the remeshing step, it is necessary to specify a target resolution in terms of vertex count. We chose a nominal target of 7,500 vertices, which offers a good compromise between preserving geometric detail and maintaining manageable processing times. Because it is often impractical to achieve exactly this number of vertices, we allow a tolerance range of 5,000 to 10,000 vertices. Remeshing therefore stops once the mesh falls within this range, ensuring both consistency across the dataset and robustness of the preprocessing pipeline.

The remeshing procedure was implemented using **Open3D** (version ≥ 0.18), an open-source library for 3D data processing. Open3D provides efficient tools for handling triangle meshes, including vertex and triangle access, mesh cleaning, simplification, and subdivision. Its functions for *quadric-based decimation* and *midpoint subdivision* were used to adjust mesh resolution while preserving the shape of the geometry. Additionally, **NumPy** was employed for numerical computations, nearest-neighbor searches, and vertex reprojection.

The goal of the remeshing step is to reduce the variation in mesh resolution across the dataset and limit the influence of outlier meshes. The algorithm works as follows:

1. **Vertex Count Check:** Each mesh is first checked against the target vertex range of 5,000 to 10,000 vertices (nominal target: 7,500). Meshes that are already within this range are copied without modification.
2. **Upsampling:** Meshes with fewer vertices than the minimum threshold are increased in resolution using a *feature-preserving midpoint subdivision*. New vertices are projected toward local planes or neighborhood centroids to maintain surface features and prevent excessive smoothing. This process is repeated in multiple passes until the mesh reaches the desired resolution.
3. **Downsampling:** Meshes exceeding the maximum vertex threshold are simplified using *quadric-based decimation*. The reduction is applied iteratively in multiple passes, with the reduction ratio adapted depending on how far the mesh is from the target. If the decimation stalls, a more aggressive pass is applied to ensure the mesh falls within the target range.
4. **Feature Preservation:** Sharp edges and important geometric features are identified based on dihedral angles between adjacent faces. During subdivision and decimation, these features guide the placement of new vertices and the removal of faces to retain the overall shape. Planar regions are preserved by blending vertex positions toward fitted local planes when possible.
5. **Fallback and Robustness:** In rare cases where standard upsampling fails, a single subdivision pass followed by light decimation is used. Mesh cleaning operations, such as removal of degenerate triangles and duplicated vertices, are applied throughout to maintain mesh quality.

This approach ensures that all meshes in the dataset have a comparable number of vertices while keeping their geometric features intact, which facilitates consistent feature extraction and object retrieval.

6.4 Step 2.4: Checking the resampling

We now evaluate the effect of the remeshing algorithm on the dataset. Again, a practical way to assess this is by examining the distribution of vertex counts across all meshes. Figure 11 shows the resulting histogram. The plot indicates that the variance in vertex counts has been substantially reduced compared to the original data. The majority of meshes now fall within the intended target range, and extreme outliers have largely been removed or adjusted.

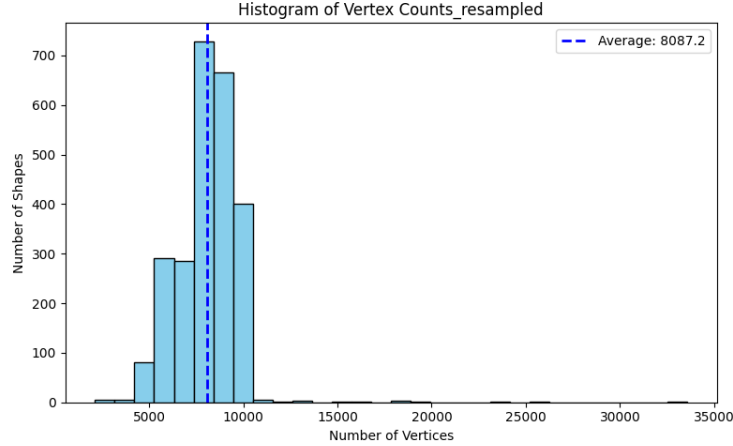


Figure 11: **Distribution of vertex counts.** Histogram of the number of vertices per shape. The horizontal axis shows vertex count (bins of approximately 3,333 vertices), and the vertical axis shows the number of shapes in each bin. The dotted line indicates the mean vertex count.

In addition, we have generated figures illustrating both the compliance rate and the distribution of meshes across all size categories: very small, small, target range, large, and very large. These comparisons, before and after resampling, are shown in Figure 12. The compliance rate has improved considerably, from 14.8% to 98.8%. While some meshes remain outside the target range, they are generally only slightly above or below the desired limits. Before resampling, the majority of meshes fell into the very small or small categories, with only a few within the target range and some extreme outliers in the very large category. The resampling step has successfully shifted most meshes into the target range, reducing the prevalence of undersized meshes and eliminating extreme outliers.

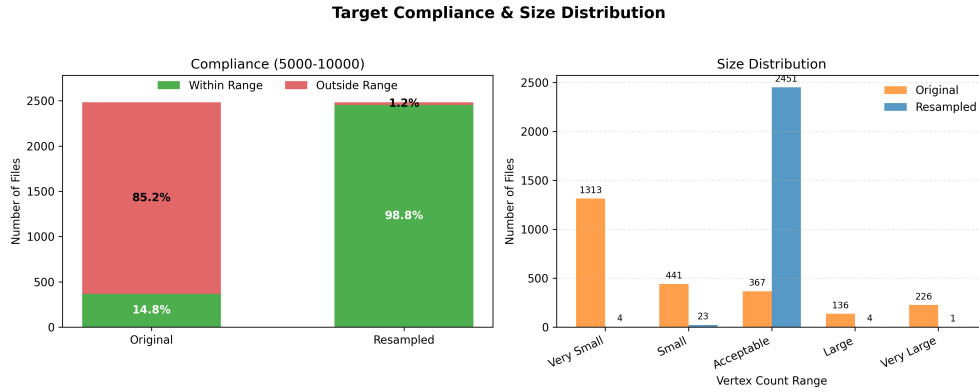


Figure 12: **(A) Target compliance and (B) Size distribution.** (A) This plot shows the proportion of meshes that fall within the acceptable target range of 5,000–10,000 vertices before and after resampling. (B) This plot shows the distribution of meshes across all size categories before and after resampling: very small (0–2,000 vertices), small (2,000–5,000), acceptable (5,000–10,000), large (10,000–15,000), and very large (15,000+).

Although the resampling pipeline substantially improved uniformity in vertex counts, a small number of meshes still fall outside of the target range of 5,000–10,000 vertices. This outcome can be explained by the

nature of the subdivision and simplification algorithms used. Subdivision increases the number of vertices in large multiplicative steps (approximately by a factor of four for each pass). As a result, meshes that start well below the lower bound often overshoot into the higher range. Simplification is then applied to reduce these meshes, but the algorithm is constrained by geometry preservation: it avoids collapsing important edges or thin structures, and therefore does not always converge exactly within the target window. Similarly, meshes that were already close to the lower bound are sometimes left unchanged, as subdivision would push them unnecessarily far above the acceptable range.

Further attempts to enforce exact compliance would require mesh-specific adjustments, which would be computationally expensive and risk degrading geometric fidelity. The current procedure achieves compliance for approximately 98% of the dataset, which we consider sufficient for our purposes. The small number of outliers does not meaningfully affect downstream tasks and avoiding excessive processing helps maintain higher shape quality overall.

It should also be noted that resampling occasionally introduces some surface artifacts. For instance, some meshes show a rippling effect along curved surfaces, such as the body of a bottle, or slight deformations in thin structures, such as the wings of an airplane no longer being perfectly straight (see Figures 13 and 14). These cases illustrate the trade-off between vertex count normalization and preservation of fine-grained geometry. While not ideal, such artifacts are limited in scope and do not compromise the overall usability of the dataset.

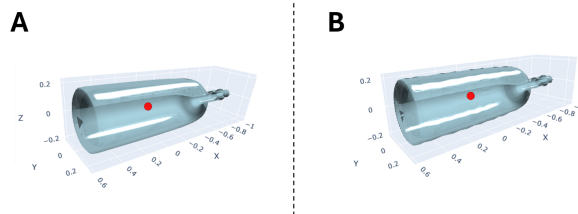


Figure 13: **Example of rippling effect after remeshing process.** (A) The original *Bottle* object (D00347.obj), consisting of 2,426 vertices and 4,438 faces. (B) The resampled *Bottle* object, now consisting of 8,624 vertices and 17,184 faces. A slight rippling effect can be observed on the along the edge of the bottle as a result of the remeshing process.

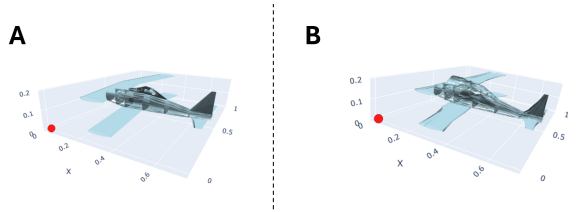


Figure 14: **Example of deformation of straight surfaces after remeshing process.** (A) The original *Biplane* object (m1131.obj), consisting of 482 vertices and 768 faces. (B) The resampled *Biplane* object, now consisting of 6,530 vertices and 12,288 faces. The previously straight wings of the airplane now have a slight deformation as a result of the remeshing process.

6.5 Step 2.5: Normalizing shapes

Having reduced the variance in mesh resolution, we next address other characteristics of the dataset that require normalization. In this substep, we translate each object so that its barycenter is positioned at the origin (0,0,0), and uniformly scale it to fit within a unit cube.

To achieve this, we first calculate the mass barycenter of the mesh, which corresponds to the center of mass under the assumption that each face has uniform density. Instead of simply averaging all vertices, we compute an area-weighted average of the centroids of all triangular faces.

For a triangular mesh $M = (V, F)$ with vertices $V = \{v_1, \dots, v_n\}$, $v_i \in \mathbb{R}^3$, and faces $F = \{f_1, \dots, f_m\}$, each face $f_j = (v_{j1}, v_{j2}, v_{j3})$ contributes as follows:

$$c(f_j) = \frac{1}{3}(v_{j1} + v_{j2} + v_{j3}),$$

$$A(f_j) = \frac{1}{2} \|(v_{j2} - v_{j1}) \times (v_{j3} - v_{j1})\|,$$

where $c(f_j)$ is the centroid of face f_j and $A(f_j)$ its area. The mass barycenter of the full mesh is then given by

$$b(M) = \frac{\sum_{j=1}^m A(f_j) c(f_j)}{\sum_{j=1}^m A(f_j)}.$$

In the degenerate case where $\sum_{j=1}^m A(f_j) = 0$ (e.g., if no valid faces are present), we fall back to the simple geometric centroid of the vertices:

$$b(M) = \frac{1}{n} \sum_{i=1}^n v_i.$$

This ensures a well-defined barycenter for all meshes in the dataset.

We have calculated the offset of the barycenter to the origin of the space before applying the translation normalization. The results can be seen in Figure 15. We observe a large variance in this offset over the dataset.

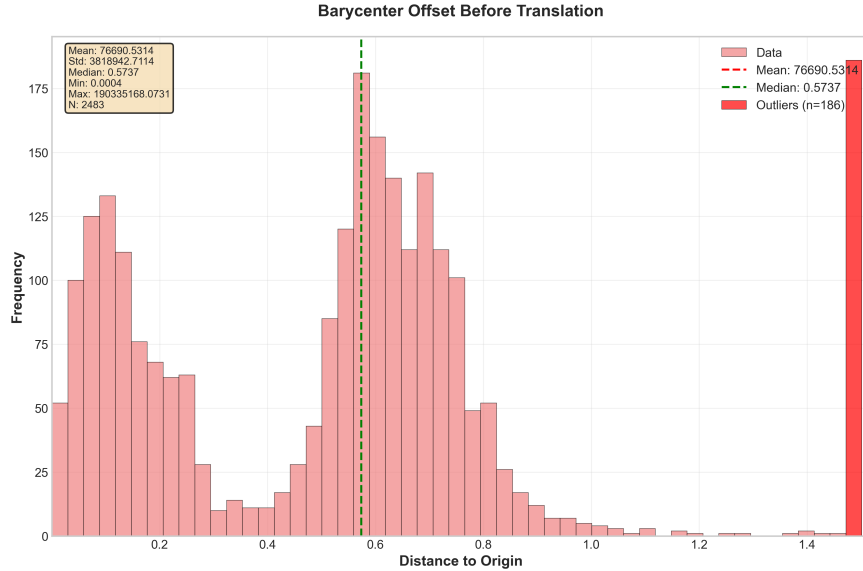


Figure 15: **Distribution of barycenter offsets before normalization.** Histogram of the Euclidean distance between each mesh barycenter and the origin. The horizontal axis shows the distance (binned), while the vertical axis shows the number of shapes. The dotted line indicates the mean offset. The red bin indicates the outliers.

Using the earlier described barycenter calculation, we translate all objects such that their barycenter is located at the origin of the space. After applying this normalization step, we again analyzed the offset of the barycenter relative to the origin. The resulting histogram can be found in Figure 16.

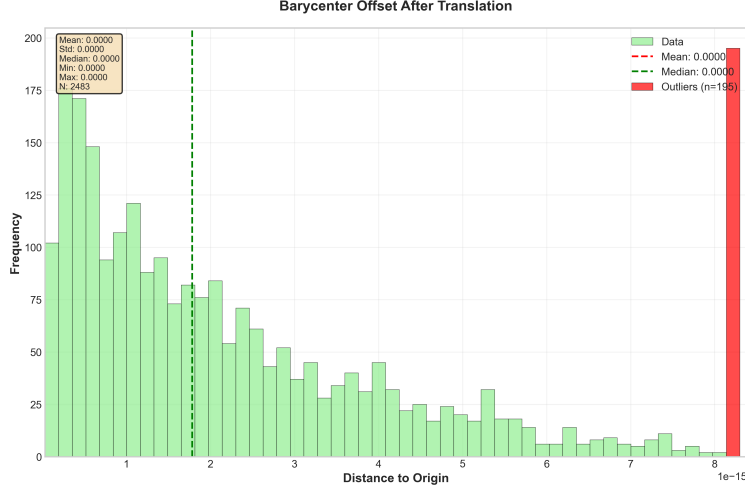


Figure 16: **Distribution of barycenter offsets after normalization.** Histogram of the Euclidean distance between each mesh barycenter and the origin. The horizontal axis shows the distance (binned), while the vertical axis shows the number of shapes. The dotted line indicates the mean offset, which is now centered around zero. (Note the small scale of the horizontal axis.)

We can see that the barycenters of all objects are now correctly positioned at the origin of the space, with even the outliers falling well below an offset of $1e-4$.

Following the translation, we normalize the scale of each mesh. Formally, we compute the axis-aligned bounding box of a mesh M , defined as

$$B(M) = [\mathbf{v}_{\min}, \mathbf{v}_{\max}], \quad \mathbf{v}_{\min} = (\min_i v_i^x, \min_i v_i^y, \min_i v_i^z), \quad \mathbf{v}_{\max} = (\max_i v_i^x, \max_i v_i^y, \max_i v_i^z).$$

We then determine the bounding box side lengths

$$\mathbf{s} = \mathbf{v}_{\max} - \mathbf{v}_{\min},$$

and take the largest dimension

$$s_{\max} = \max(s_x, s_y, s_z).$$

To enforce uniform scaling, each vertex is scaled by the factor

$$\alpha = \frac{1}{s_{\max}},$$

such that the longest side of the bounding box is mapped to length 1.0. This guarantees that every object is contained within a unit cube $[-0.5, 0.5]^3$.

In Figure 17 we show a histogram of the bounding box longest side length before this normalization step. We can observe quite a large variance in this characteristic over the dataset. After applying the scaling procedure, the distribution collapses to a single value of 1.0, as illustrated in Figure 18.

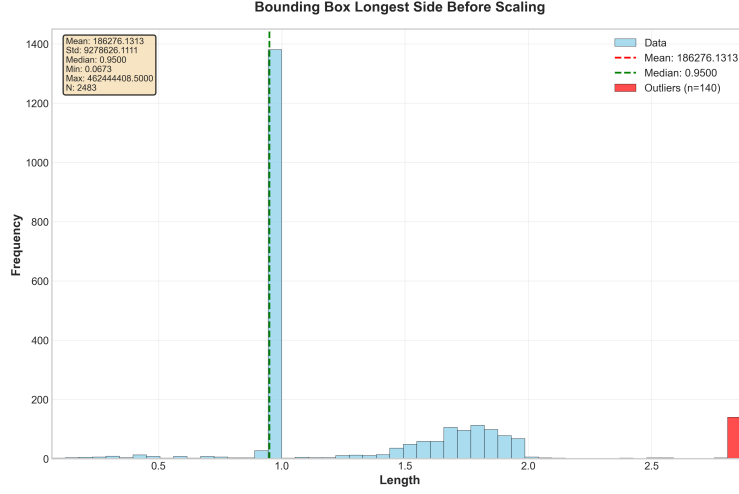


Figure 17: **Distribution of bounding box sizes before normalization.** Histogram of the longest side length of the bounding box per shape. The horizontal axis shows side length, while the vertical axis shows the number of shapes.

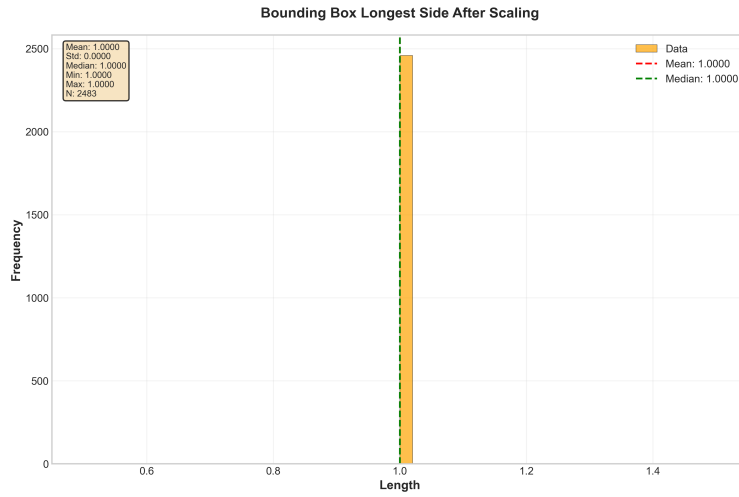


Figure 18: **Distribution of bounding box sizes after normalization.** Histogram of the longest side length of the bounding box per shape. All objects have been normalized to fit inside the unit cube, leading to uniform scaling.

Together, these two normalization steps — translating meshes to the origin and scaling them to the unit cube — ensure that all objects are centered, uniformly scaled, and thus more suitable to be compared directly. This eliminates biases due to arbitrary positioning or size differences in the raw dataset, enabling fairer and more consistent feature extraction in subsequent stages.

7 Step 3: Feature extraction

7.1 Step 3.1: Full normalization

7.2 Step 3.2: 3D shape descriptors

8 Step 4: Querying

9 Step 5: Scalability

10 Step 6: Evaluation

References

- [1] Philip Shilane et al. “The Princeton Shape Benchmark”. In: *Proceedings of the Shape Modeling International Conference*. IEEE, 2004, pp. 167–178. DOI: [10.1109/SMI.2004.1314504](https://doi.org/10.1109/SMI.2004.1314504). URL: <https://shape.cs.princeton.edu/benchmark/>.
- [2] Paul Bourke. *Object Files (.obj)*. <http://paulbourke.net/dataformats/obj/>. Accessed: December 2024. 1999.
- [3] Plotly Technologies Inc. *Dash: A web application framework for Python*. <https://plotly.com/dash>. Version 2.0 and above. 2017.
- [4] Plotly Technologies Inc. *Plotly.py: An interactive, open-source, and browser-based graphing library for Python*. <https://plotly.com/python>. Version 5.0 and above. 2015.
- [5] Wes McKinney et al. “pandas: a foundational Python library for data analysis and statistics”. In: *Python for high performance scientific computing*. Vol. 14. 9. Austin, TX. 2011, pp. 1–9.
- [6] Charles R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [7] John Deacon. “Model-view-controller (mvc) architecture”. In: *Online* [Citado em: 10 de março de 2006.] <http://www.jdl.co.uk/briefings/MVC.pdf> 28 (2009), p. 61.
- [8] Rufu Qin et al. “Web-based 3D visualization framework for time-varying and large-volume oceanic forecasting data using open-source technologies”. In: *Environmental Modelling & Software* 135 (2021), p. 104908.
- [9] Plotly Technologies Inc. *Plotly Graph Objects Python Library*. 2025. URL: <https://plotly.com/python/graph-objects/>.
- [10] Mario Botsch et al. *Polygon Mesh Processing*. CRC Press, 2010. URL: <https://www.pmp-book.org/>.